

Anil Seth



Python as an Erlang Distributed Node

Erlang is a programming language usually associated with banking, e-commerce, computer telephony and telecommunications. And Python is a powerful programming language used in a wide variety of application domains. This article shows readers how to integrate the two.

Erlang is a superb option for developers to exploit multiple processors or cores (*Open Source For You*, December 2013). In Erlang, it is easy to create processes that communicate with each other using asynchronous communication. Erlang provides libraries for C and Java to communicate with it, as if the latter two were a distributed node in the Erlang universe. Of course, it is more likely that you already have a C or a Java application and would like to exploit the advantages of Erlang to enhance the functionality of your existing application.

In this article, let's explore how to use `py_interface` (http://www.lysator.liu.se/~tab/erlang/py_interface) to integrate a Python application with an Erlang application. As an illustration, let's use the example of concurrent programming in http://www.erlang.org/doc/getting_started/conc_prog.html and create a Python client that communicates with the Erlang messenger server.

The Erlang server receives a message for a logon, logoff or a 'send message' request. The Erlang code would be as follows, where the `User_List` contains the names of the signed-in users and their addresses:

```
server(User_List) ->
receive
    {From, logon, Name} ->
        New_User_List = server_logon(From, Name, User_List),
        server(New_User_List);
    {From, logoff} ->
        New_User_List = server_logoff(From, User_List),
        server(New_User_List);
    {From, message_to, To, Message} ->
        server_transfer(From, To, Message, User_List),
        server(User_List)
end.
```

The above code calls various Erlang functions to do the work. Since the code is available on the Erlang website and our focus is on Python integrating with Erlang, minimal Erlang code is included here.

Your Python code will first have to become an Erlang node and listen for any incoming events. The comments in the code explain the steps needed.

```
import types
```

```
from py_interface import erl_common
from py_interface import erl_term
from py_interface import erl_node
from py_interface import erl_opts
from py_interface import erl_eventhandler
```

```
def main():
    global mb
    hostName = "netbook"
    ownNodeName = "pyclient"
    cookie = "erlang.cookie"
    #Create Erl node and Publish it
    n = erl_node.ErlNode(ownNodeName, erl_opts.
ErlNodeOpts(cookie=cookie))
    n.Publish()
    # Create a mailbox for receiving messages
    mb = n.CreateMBox(MBoxCallback)
    # Register mbox with a symbolic name - not used in
this example
    mb.RegisterName("Py")
    # Create and start an eventhandler loop
    evhand = erl_eventhandler.GetEventHandler()
    timeout = 1
    # Add a timer event for the logon message to be sent
    evhand.AddTimerEvent(timeout, logon_callback)
    evhand.Loop()
```

The above code includes a timeout event after one second, which will call the function to send the logon request. It would be preferable to create two separate threads/processes for incoming and outgoing messages. However, the idea here is to focus on the `py_interface` functions only. A simple timeout illustrates the `py_interface` functions just as well.

The Erlang message will be a tuple (not a list) with three terms:

- Sender's ID—the process ID or the tuple of the registration name and the node name
- Logon Erlang token
- Name—Erlang token for consistency